FUNDAMENTOS DE COMPUTADORES · 1.º GII · PRÁCTICAS

Guía de prácticas $ _

Los boletines de los seis temas explicados: qué se hace en cada sesión, lo que tienes que saber y todos los ejercicios resueltos paso a paso. Hay dos herramientas para comprobar tus cálculos: un mini-okteta y una calculadora de redes.

1

Tema 1

Estructura general de un PC

Boletín 1 del tema 1: los componentes de un PC de sobremesa, cómo se conectan a la placa base y cómo se desmonta y monta.

1.1 Qué se hace en la sesión

Se estudia un PC desmontado por el profesor: repaso de Von Neumann, explicación de cada componente y su conexión a la placa base, inspección directa y consulta del manual de la placa.

La **placa base** es el elemento central: a ella se conectan la CPU, la memoria, los discos, la fuente y la refrigeración, así que condiciona buena parte de la funcionalidad y del rendimiento.

1.2 Parámetros que definen una placa base

Parámetro	Qué hay que saber
Factor de forma	Medidas, agujeros y posición de conectores: restringe la caja. **ATX** en PC; WTX (mayor) en servidores; en portátiles no hay estándar.
CPUs soportadas	Las fija el **zócalo** (socket 939 → AMD Athlon 64; socket 775 → Pentium IV, Pentium D, Core) y las frecuencias admitidas.
Memoria	Tipo (DDR, DDR2...DDR4), frecuencia, nº de ranuras, capacidad máxima, canales (ranuras por parejas).
Chipset	Todo flujo entre dos elementos pasa por él. **Memory Controller Hub** (northbridge): CPU, gráfica, memoria y el ICH. **I/O Controller Hub** (southbridge): todo lo demás. AMD y después Intel integraron el MCH en la CPU.
Ranuras de expansión	**PCI**: 32 bits a 33 MHz, un dato por ciclo → ≈ 133 MB/s pico. **PCI Express**: punto a punto, serie, en canales xN; la x16 (8, 16 o 32 GB/s según versión) se usa para la gráfica.
Almacenamiento	**PATA/ATA/IDE**: paralelo, hasta 133 MB/s. **SATA**: serie punto a punto, 150 MB/s al principio y ≈ 2 GB/s en la 3.2. HDD (magnético) frente a SSD (más rápido y caro por bit).
Audio y red integrados	Reducen el coste y el consumo.
Puertos	LAN RJ-45, audio, VGA/DVI/HDMI, PS/2 y, sobre todo, USB (3.x: varios Gbit/s).
BIOS / UEFI	Programa en un chip (Flash BIOS) que arranca el PC y maneja E/S básica; guarda la configuración en la CMOS, alimentada por la pila. **Actualizarla es la operación de mantenimiento más crítica.** UEFI la está sustituyendo.

1.3 Desmontaje y montaje

⚠ Antes de empezar

- Siempre con el PC **desenchufado**.
- Documenta cables y jumpers antes de tocarlos.
- Descárgate de electricidad estática (muñequera antiestática).
- Anota las características (placa, procesador, BIOS, ranuras): están en el manual de la placa.

1. Retirar las tarjetas PCI (gráfica, red, sonido): se desatornillan en el lateral.
2. Desmontar discos y unidades ópticas, fijándose en la banda roja del cable y en el conector (IDE1, IDE2, FLOPPY).
3. Retirar los módulos de memoria liberando las pestañas laterales.
4. Desmontar el ventilador de la CPU.
5. Levantar la palanca del zócalo y extraer la CPU fijándose en la marca del pin 1 (en un PC operativo, limpiar la pasta térmica).
6. Documentar y retirar los cables de la placa al chasis (LED, speaker...).
7. Retirar el cable de alimentación de la placa.
8. Volver a montarlo todo como estaba.

1.4 Glosario

Sigla	Significado
CPU / GPU	Unidad central de proceso / de procesamiento gráfico
RAM / ROM	Memoria de acceso aleatorio / de sólo lectura
PCI / PCIe	Interconexión de componentes periféricos / su versión serie, PCI Express
AGP	Puerto de gráficos avanzado (obsoleto)
ATA, PATA, IDE	Interfaz paralela de discos (sinónimos)
SATA	ATA serie, más moderno
LAN	Red de área local (conector RJ-45)
VGA / DVI / HDMI	Vídeo analógico / digital / digital de alta definición
HDD / SSD	Disco magnético / de estado sólido
CMOS	Memoria con la configuración de la BIOS
BIOS / UEFI	Firmware de arranque / su sustituto

Ejercicio 1.1 · **Ancho de banda del bus PCI**

El bus PCI más usado tiene 32 bits, funciona a 33 MHz y transfiere un dato por ciclo. Calcula su tasa de transferencia pico en MB/s.

1. Ancho de banda = frecuencia × ancho = $33 \cdot 10^6$ ciclos/s × 32 bits.
2. = $1056 \cdot 10^6$ bit/s = 1056 Mbit/s.
3. En bytes: 1056 / 8 = **132 MB/s** (el boletín lo redondea a 133, con 33,33 MHz).

RESULTADO ≈ 133 MB/s

Ejercicio 1.2 · ¿Quién controla qué?

Di si cada elemento depende del Memory Controller Hub (northbridge) o del I/O Controller Hub (southbridge): a) módulos de RAM; b) discos SATA; c) tarjeta gráfica PCIe x16; d) USB; e) audio integrado.

Elemento	Chip
a) RAM	Northbridge (MCH)
b) SATA	Southbridge (ICH)
c) Gráfica PCIe x16	Northbridge (MCH)
d) USB	Southbridge
e) Audio	Southbridge

Regla: lo de alta velocidad (memoria y gráfica) va al norte; el resto de E/S, al sur. En placas actuales el norte está dentro de la CPU.

Ejercicio 1.3 · Orden de desmontaje

Ordena: (A) extraer la CPU; (B) retirar las tarjetas PCI; (C) quitar el cable de alimentación de la placa; (D) retirar la memoria; (E) desmontar el ventilador de la CPU; (F) desmontar los discos.

ORDEN B → F → D → E → A → C

Primero lo que estorba (tarjetas y discos), después memoria, disipador y CPU, y al final los cables de la placa y su alimentación.

2

Tema 2

Decodificación de ficheros con okteta

Boletines 1 y 2 del tema 2: interpretar y crear a mano ficheros binarios con enteros, reales y texto, imágenes PPM, HTML y documentos ODF.

2.1 La herramienta okteta

Editor de ficheros binarios con tres zonas: el **contenido plano** (cada byte en hexadecimal), la **interpretación como caracteres** y la **columna de direcciones** (offset del primer byte de cada fila; el del cursor sale abajo a la izquierda). La **tabla de decodificación** interpreta el byte del cursor y los siguientes como enteros de 8/16/32/64 bits con y sin signo, float y double.

> ⚠ **Little y big endian**
>
> Big endian guarda el byte menos significativo **al final** (dirección más alta); little endian, **al principio**. Hay que decirle a okteta cuál se usa. **Todos los ficheros del boletín son little endian.**

2.2 Ejemplo: salidac.bin

22 bytes con una cadena de 4 caracteres, un entero de 16 bits, uno de 32, un float y un double:

salidac.bin

```
48 6F 6C 61 FA FF 00 00 01 00 00 00 00 40 52 B8 1E 85 EB 51 12 40
```

Offset	Bytes	Tipo	Valor
0x00	48 6F 6C 61	4 caracteres ASCII	«Hola»
0x04	FA FF	int16 (C2)	0xFFFA = **−6**
0x06	00 00 01 00	int32	0x00010000 = **65536**
0x0A	00 00 00 40	float	0x40000000 = **2**
0x0E	52 B8 1E 85 EB 51 12 40	double	0x401251EB851EB852 = **4,58**

Método: coloca el cursor en el primer byte de cada dato y lee la tabla de decodificación. A mano: invierte el orden de los bytes (little endian) y decodifica (FA FF → 0xFFFA → C2 → −6).

2.3 Herramienta: bytes ↔ valores INTERACTIVO

Compruébalo todo aquí: codifica un valor en bytes como harías en okteta, o pega bytes y mira cómo se interpretan desde un desplazamiento.

2.4 Ejercicios del boletín 1

Ejercicio 2.1 · **Interpretar datos.bin**

datos.bin contiene, por este orden: una cadena de 6 caracteres, un double, un int32, un float y un int16 (little endian). ¿En qué offset empieza cada dato?

Dato	Bytes	Offset
Cadena	6	0x00–0x05
double	8	0x06–0x0D
int32	4	0x0E–0x11
float	4	0x12–0x15
int16	2	0x16–0x17

Total 24 bytes. Coloca el cursor en cada offset y lee en la tabla de decodificación el tipo correspondiente (los valores dependen del fichero del Aula Virtual).

Ejercicio 2.2 · **Escribir PRUEBA, 8.75, −140506, 5 y 36**

Modifica datos.bin para que guarde «PRUEBA», el double 8,75, el int32 −140506, el float 5 y el int16 36.

Cálculos

- «PRUEBA» en ASCII: `50 52 55 45 42 41`.
- 8,75 = 1000,11)2 = 1,00011 × 2^3; e = 3 + 1023 = 1026 = 100 0000 0010 → 0 10000000010 0001 1000 0… = 0x4021800000000000 → little endian `00 00 00 00 00 80 21 40`.
- −140506: 140506 = 0x000224DA → C2: 0xFFFFFFFF − 0x000224DA + 1 = 0xFFFDDB26 → `26 DB FD FF`.
- 5 = 101 = 1,01 × 2^2; e = 129 = 10000001 → 0x40A00000 → `00 00 A0 40`.
- 36 = 0x0024 → `24 00`.

datos.bin modificado (24 bytes)

```
50 52 55 45 42 41 00 00 00 00 00 80 21 40 26 DB FD FF 00 00 A0 40 24 00
```

2.5 Imágenes PPM

PPM (Portable Pixel Map) es ineficiente (sin compresión) pero muy sencillo. Estructura:

1. Número mágico `P6`.
2. Delimitadores (espacio, tabulador, CR o, normalmente, `0A`).
3. Anchura y altura en decimal con caracteres ASCII.
4. Valor máximo de color (0 < máx < 65536) en ASCII.
5. Los píxeles de arriba abajo y de izquierda a derecha: tripletas R, G, B en binario puro (si máx > 255, 2 bytes por color, el más significativo primero).
6. Comentarios desde `#` hasta un delimitador.

Imagen 4×4 con marco blanco y fondo negro (máx 255): cada `FFFFFF` es un píxel blanco y cada `000000`, uno negro.

```
FFFFFF FFFFFF FFFFFF FFFFFF
FFFFFF 000000 000000 FFFFFF
FFFFFF 000000 000000 FFFFFF
FFFFFF FFFFFF FFFFFF FFFFFF
```

En blanco.ppm okteta muestra P6, 33×33 píxeles, máximo 255 y todos los píxeles `FF FF FF`. La cabecera `P6␊33 33␊255␊` ocupa **13 bytes**, así que los píxeles empiezan en el offset 0x0D.

2.6 HTML y Open Document

HTML: etiquetas entre < y > con atributos opcionales. Cabecera (`<HEAD>`: título, metadatos) y cuerpo (`<BODY>`). `<B>` negrita, `<I>` cursiva, `<P>` párrafo, `<BR>` salto de línea, `<A HREF="…">` enlace; `á` = á.

Open Document (ODF): estándar abierto para documentos ofimáticos. Un .odt/.ods es un **ZIP** con `content.xml` (el contenido), `styles.xml` (estilos), `meta.xml` (autor, fechas) y `settings.xml` (zoom, cursor); las imágenes van en un directorio aparte.

2.7 Ejercicios del boletín 2

Ejercicio 2.3 · Fórmula de la posición de un píxel

Sabiendo que los píxeles empiezan en el offset *desp*, ¿en qué posición empiezan los bytes R, G, B del píxel (x, y)?

Los píxeles van fila a fila; antes del píxel (x, y) hay y filas completas de *ancho* píxeles y x píxeles más, y cada píxel ocupa 3 bytes (máx ≤ 255):

```
pos(x, y) = desp + 3 · (y · ancho + x)
```

con x = columna e y = fila, contando desde 0. G está en pos + 1 y B en pos + 2.

Ejercicio 2.4 · Cuadrado azul de 3×3 arriba a la izquierda

Modifica blanco.ppm (33×33) para dibujar un cuadrado azul de 3×3 en la esquina superior izquierda.

Azul puro = `00 00 FF`. Con desp = 13 y ancho = 33, cada fila del cuadrado son 9 bytes seguidos:

Fila y	Offset inicial	Bytes a escribir
0	13 = 0x0D	00 00 FF × 3
1	13 + 99 = 112 = 0x70	00 00 FF × 3
2	13 + 198 = 211 = 0xD3	00 00 FF × 3

(Cada fila completa ocupa 33 · 3 = 99 bytes.)

Ejercicio 2.5 · Cuadrado negro de 3×3 en el centro

Añade un cuadrado negro de 3×3 justo en el centro de la imagen.

El píxel central es (16, 16), así que el cuadrado ocupa x, y ∈ {15, 16, 17}. Negro = `00 00 00`:

Fila y	Offset de (15, y)	Bytes
15	13 + 3·(33·15 + 15) = 1543 = 0x607	9 × 00
16	1642 = 0x66A	9 × 00
17	1741 = 0x6CD	9 × 00

El fichero mide 13 + 33·33·3 = 3280 bytes.

Ejercicio 2.6 · Mi primera web

Crea una página titulada «Mi primera Web en Fundamentos de Computadores» con los nombres en negrita, los DNI en cursiva, una línea en blanco y un país enlazado a su página de la Wikipedia.

web.html

```
<HTML>
<HEAD>
<TITLE>Mi primera Web en Fundamentos de Computadores</TITLE>
</HEAD>
<BODY>
<P><B>Ana Garc&iacute;a L&oacute;pez, Luis Mart&iacute;nez Ruiz</B></P>
<P><I>12345678A, 87654321B</I></P>
<BR>
<P><A HREF="https://es.wikipedia.org/wiki/Portugal">Portugal</A></P>
</BODY>
</HTML>
```

Ábrela con el navegador y comprueba el título de la pestaña. Las tildes van con entidades (`í`) para que salgan bien con cualquier codificación.

Ejercicio 2.7 · **Dentro de un .odt y un .ods**

Crea con LibreOffice un texto (dirección postal con la ciudad en negrita) y una hoja de cálculo con dos columnas y su suma. Descomprímelos y busca tu contenido.

```
$ mkdir odt && cd odt && unzip ../direccion.odt
$ ls            # content.xml  meta.xml  settings.xml  styles.xml  META-
INF/ ...
$ less content.xml
```

El texto aparece en `content.xml` dentro de `<text:p>`; la ciudad en negrita va en un `<text:span>` con un estilo automático que define `fo:font-weight="bold"`. En la hoja, cada celda es un `<table:table-cell>` y la suma guarda la fórmula en `table:formula="of:=[.A1]+[.B1]"` además del valor calculado.

3

Tema 3

Sumador de n bits y exhibidor BCD

Boletines 1 y 2 del tema 3: diseño y simulación en TkGate de un sumador y de un decodificador para un display de 7 segmentos.

3.1 Semisumador y sumador completo

Se diseña el sumador como se suma «con lápiz y papel»: bit a bit, con el acarreo del bit anterior.

A_i	B_i	R_i	C_{i+1}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Semisumador: $R_i = A_i \oplus B_i$ (XOR) y $C_{i+1} = A_i \cdot B_i$ (AND). No tiene en cuenta el acarreo de entrada.

Sumador completo con dos semisumadores: el primero suma A y B; el segundo suma ese resultado con C_i; el acarreo de salida es la OR de los dos acarreos.

A_i	B_i	C_i	R_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

R_i

C \ AB	00	01	11	10
0	0 0	1 2	0 6	1 4
1	1 1	0 3	1 7	0 5

C_{i+1}

C \ AB	00	01	11	10
0	0 0	0 2	1 6	0 4
1	0 1	1 3	1 7	1 5

En los mapas (columnas A_iB_i, filas C_i) los unos de R no se pueden agrupar (patrón de tablero): quedan los 4 minitérminos, que son la XOR de tres entradas. Los de C forman tres parejas, todas esenciales:

$$R_i = A'B'C + A'BC' + AB'C' + ABC = A \oplus B \oplus C_i$$

$$C_{i+1} = A \cdot B + A \cdot C_i + B \cdot C_i$$

3.2 Sumador serie de n bits y simulación

Se encadenan n sumadores de 1 bit: el acarreo de salida de cada uno es el de entrada del siguiente, y el C_{in} del bit 0 va a **tierra** (0). Es el **sumador serie** o con propagación de acarreo. Sirve también para restar en C2.

Problema: cada bit espera al acarreo del anterior y los retardos se **acumulan** (64 sumadores en serie para 64 bits). Los procesadores usan sumadores con **acarreo adelantado** (carry lookahead).

En TkGate (sumador.v)

1. Carga sumador.v. En la vista Árbol, **sumador4bits** muestra los 4 sumadores en cascada (a0–a3, b0–b3, salidas c0–c3 y cout; cin a tierra).
2. **sumador** es el de 1 bit (a, b, c → r, ci) hecho con los módulos **acarreo** y **resultado** a nivel de puertas.
3. En **main**, dos interruptores DIP dan los operandos en hexadecimal, un display de 7 segmentos muestra el resultado y un LED el acarreo (se enciende si la suma es > 15).
4. Pestaña de simulación → play → clic derecho en los DIP para cambiarlos y comprobar resultados.

Ejercicio 3.1 · **Sumar 1011 + 0110 con el sumador de 4 bits**

Sigue el acarreo bit a bit. ¿Qué muestran el display y el LED de acarreo?

Bit	A	B	C_{in}	R	C_{out}
0	1	0	0	1	0
1	1	1	0	0	1
2	0	1	1	0	1
3	1	0	1	0	1

RESULTADO R = 0001 y C_{out} = 1

11 + 6 = 17 = 1 0001: el display muestra **1** y el LED se **enciende** porque la suma pasa de 15.

3.3 El exhibidor BCD de 7 segmentos

Entrada de 4 bits A3A2A1A0: los dígitos 0–9 en BCD, la **E** de error (1110) y el signo **menos** (1111). Las entradas 1010–1101 no se usan: son **indiferencias (X)**. Cada segmento a–g es una función de las cuatro entradas y se implementa con una técnica distinta.

N°	A3	A2	A1	A0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
A	1	0	1	0	X	X	X	X	X	X	X
B	1	0	1	1	X	X	X	X	X	X	X
C	1	1	0	0	X	X	X	X	X	X	X
D	1	1	0	1	X	X	X	X	X	X	X
E (error)	1	1	1	0	1	0	0	1	1	1	1
- (menos)	1	1	1	1	0	0	0	0	0	0	1

Segmento	Técnica
a	Suma de productos (forma disyuntiva)
b	Producto de sumas (forma conjuntiva)
c	Sólo puertas NAND
d	Sólo puertas NOR
e	MUX 8 a 1 (+ un inversor)
f, g	Ilustrativos: decodificador 3 a 8 y demultiplexor (no se exigen)

En los mapas, columnas A3A2 y filas A1A0; el número pequeño es el índice de la celda.

3.4 Segmento a: suma de productos

a

A1A0 \ A3A2	00	01	11	10
00	1 0	0 4	X 12	1 8
01	0 1	1 5	X 13	1 9
11	1 3	1 7	0 15	X 11
10	1 2	1 6	1 14	X 10

Implicantes primos: A2'A0', A3'A1, A2A1'A0, A1A0', A3A2', A3A1', A2'A1, A3'A2A0, A3A0'. El único **esencial** es **A2'·A0'** (cubre el 0 y el 8). Una cubierta mínima, la de las transparencias:

```
a = A2'·A0' + A3'·A1 + A2·A1'·A0 + A1·A0' + A3·A2'
```

Circuito: un primer nivel de puertas AND (una por producto) y una OR de cinco entradas.

3.5 Segmentos b, c, d y e resueltos

Ejercicio 3.2 · Segmento b como producto de sumas

Minimiza b agrupando ceros y dibuja el circuito OR-AND.

b (agrupamos los ceros)

A1A0 \ A3A2	00	01	11	10
00	1 0	1 4	X 12	1 8
01	1 1	0 5	X 13	1 9
11	1 3	1 7	0 15	X 11
10	1 2	0 6	0 14	X 10

1. Ceros: 5, 6, 14, 15. Indiferencias: 10–13.
2. {10, 11, 14, 15} → A3·A1 (cambian A2 y A0).
3. {6, 14} → A2·A1·A0'.
4. {5, 13} → A2·A1'·A0 (el 13 es X).
5. b' = A3A1 + A2A1A0' + A2A1'A0 y, por De Morgan, cada grupo da una suma con las variables **negadas si valen 1**:

```
b = (A3' + A1') · (A2' + A1' + A0) · (A2' + A1 + A0')
```

Circuito: tres OR (con las entradas negadas que hagan falta) y una AND de tres entradas. Comprobación: en la fila 5 (0101) la tercera suma vale 0 + 0 + 0 = 0 → b = 0 ✓.

Ejercicio 3.3 · **Segmento c sólo con NAND**

Minimiza c por unos e impleméntalo sólo con puertas NAND.

c

A1A0 \ A3A2	00	01	11	10
00	1 0	1 4	X 12	1 8
01	1 1	1 5	X 13	1 9
11	1 3	1 7	0 15	X 11
10	0 2	1 6	0 14	X 10

1. Ceros sólo en 2, 14 y 15.
2. {0,1,4,5,8,9,12,13} → **A1'** (grupo de 8 con las X 12 y 13).
3. Faltan 3, 6 y 7: {4,5,6,7} → **A3'·A2** cubre 6 y 7; {1,3,5,7} → **A3'·A0** cubre el 3.

```
c = A1' + A3'·A2 + A3'·A0
```

Doble negación y De Morgan: c = (A1 · (A3'A2)' · (A3'A0)')'. Es decir:

- Primer nivel: NAND(A3', A2) y NAND(A3', A0). A3' se saca con una NAND de entradas unidas.
- El término de una sola variable, A1', entra en la NAND final **negado**: entra A1 directamente.
- Segundo nivel: **c = NAND(A1, NAND(A3', A2), NAND(A3', A0))**.

Ejercicio 3.4 · **Segmento d sólo con NOR**

Minimiza d por ceros e impleméntalo sólo con puertas NOR.

d (agrupamos los ceros)

A1A0 \ A3A2	00	01	11	10
00	1 0	0 4	X 12	1 8
01	0 1	1 5	X 13	1 9
11	1 3	0 7	0 15	X 11
10	1 2	1 6	1 14	X 10

1. Ceros: 1, 4, 7, 15.
2. El 1 (0001) no tiene vecinos cero ni X: queda solo → (A3 + A2 + A1 + A0').
3. {4, 12} → A2A1'A0' → (A2' + A1 + A0).
4. {7, 15} → A2A1A0 → (A2' + A1' + A0').

```
d = (A3 + A2 + A1 + A0') · (A2' + A1 + A0) · (A2' + A1' + A0')
```

Con NOR los dos niveles se sustituyen directamente: **d = NOR(NOR(A3, A2, A1, A0'), NOR(A2', A1, A0), NOR(A2', A1', A0'))**. Las variables negadas se obtienen con NOR de entradas unidas.

Ejercicio 3.5 · **Segmento e con un MUX 8 a 1**

Implementa e con un multiplexor de 3 entradas de control y un inversor.

A3, A2, A1 al control; cada entrada de datos agrupa las filas A0 = 0 y A0 = 1 y recibe la **función residuo** en A0:

A3A2A1	Filas	e (A0=0, A0=1)	Entrada
000	0, 1	1, 0	I0 = A0'
001	2, 3	1, 0	I1 = A0'
010	4, 5	0, 0	I2 = 0
011	6, 7	1, 0	I3 = A0'
100	8, 9	1, 0	I4 = A0'
101	10, 11	X, X	I5 = 0 (o A0')
110	12, 13	X, X	I6 = 0 (o A0')
111	14, 15	1, 0	I7 = A0'

El inversor da A0'. (Simplificado sería e = A2'·A0' + A1·A0'.)

Ejercicio 3.6 · **Segmentos f y g (repaso)**

Aunque en clase se dan hechos con decodificador y demultiplexor, minimízalos como suma de productos.

f

A1A0 \ A3A2	00	01	11	10
00	1 0	1 4	X 12	1 8
01	0 1	1 5	X 13	1 9
11	0 3	0 7	0 15	X 11
10	0 2	1 6	1 14	X 10

g

A1A0 \ A3A2	00	01	11	10
00	0 0	1 4	X 12	1 8
01	0 1	1 5	X 13	1 9
11	1 3	0 7	1 15	X 11
10	1 2	1 6	1 14	X 10

```
f = A1'·A0' + A2·A1' + A2·A0' + A3·A1'
```

```
g = A3 + A2·A1' + A2'·A1 + A1·A0'
```

Con un decodificador 3 a 8 (o un demultiplexor con un 1 en la entrada de datos) cada salida es un minitérmino de las variables de control, y basta combinarlas con la cuarta variable, como con el MUX.

Simulación (deco7seg.v)

Carga deco7seg.v en TkGate, revisa los módulos segmentoA...segmentoF en la vista Árbol y simula con el interruptor DIP probando las entradas 0–F.

4

Tema 4

Linux desde la línea de comandos

Boletines 1 y 2 del tema 4: moverse por el sistema, ficheros y permisos, búsquedas, copias de seguridad, procesos y redirecciones. Los comandos están probados en bash.

4.1 Guía rápida del boletín

El intérprete (bash) ejecuta **órdenes internas** (las conoce él) o **programas externos** (los busca en los directorios del `PATH`). Los ejercicios se hacen en grupos de dos; en los laboratorios el usuario es `alumno` / `alumno` y el home `/home/alumno`.

Necesito…	Orden
Saber dónde estoy / moverme	`pwd`, `cd ruta`, `cd`, `cd -`
Listar	`ls -l`, `ls -a`, `ls -d .*`
Ver un fichero	`cat`, `less`, `hexdump -C`
Copiar conservando atributos	`cp -a` (o `cp -rp`)
Buscar	`find dir -name … -type f|d …`
Empaquetar	`tar czvf` / `tzvf` / `xzvf`
Permisos	`chmod 755 f`, `chmod -x f`
Procesos	`&`, `fg`, `bg`, Ctrl-C, Ctrl-Z, `ps -Af`, `kill -9`
Redirigir	`>`, `>>`, `2>`, `<`, `|`

4.2 Boletín 1 resuelto

B1 · Ejercicio 1 · Terminales de texto

Cambia a un terminal de texto, entra y prueba date, pwd, w, ls y ls -l. Cierra la sesión y vuelve al gráfico.

```
Ctrl-Alt-F3          # terminal de texto (F1-F6)
$ date               # fecha y hora
$ pwd                # /home/alumno
$ w                  # usuarios conectados y qué hacen
$ ls                 # sólo los nombres
$ ls -l              # formato largo: permisos, dueño, tamaño, fecha
$ exit
Ctrl-Alt-F7          # vuelve al entorno gráfico
```

La diferencia entre `ls` y `ls -l` es el formato largo con los metadatos de cada fichero.

B1 · Ejercicio 3 · Movimiento básico por el sistema

a) ruta actual; b) ir a /usr/bin con ruta absoluta; c) listado largo; e) sólo los que empiezan por «a»; f) volver al home; g) ir a /etc/X11 con ruta relativa; i) desde el home, listar con ruta relativa los ficheros de /usr/bin de exactamente 4 caracteres; j) igual con ruta absoluta.

```
$ pwd                       # a) /home/alumno
$ cd /usr/bin ; pwd         # b)
$ ls -l                     # c) todos son ejecutables
$ ls a*                     # e)
$ cd ; pwd                  # f) /home/alumno
$ cd ../../etc/X11 ; pwd    # g) desde /home/alumno: sube dos veces a / y
baja
$ cd                        #    vuelve al home
$ ls ../../usr/bin/????     # i) ruta relativa
$ ls /usr/bin/????          # j) ruta absoluta
```

h) Prueba el tabulador: `cd /us[Tab]/bi[Tab]` completa los nombres. d) Compara con el navegador de archivos (nautilus/konqueror).

B1 · Ejercicio 4 · Crear, copiar, mover y renombrar

a) crea pracFC y copia en él lo de /etc que empieza por g conservando atributos; b–c) compara /etc/group y pracFC/group; e–f) mira sus códigos ASCII; g) mueve group al home; i) renombra pracFC a copia_etc; j) muestra los ocultos.

```
$ mkdir pracFC
$ cp -a /etc/g* pracFC          # a) -a = -dR --preserve=all (algunos
pueden dar «Permiso denegado»)
$ ls pracFC
$ cat /etc/group pracFC/group    # b) mismo contenido
$ ls -l /etc/group pracFC/group  # c) dueño y grupo: root (con -a se
conservan)
$ hexdump -C pracFC/group        # e)
$ cd /usr
$ hexdump -C ../home/alumno/pracFC/group   # f) ruta relativa desde /usr
$ cd -                           #    vuelve al directorio anterior
$ mv pracFC/group .              # g) (o: mv pracFC/group ~)
$ cat group                      # h)
$ mv pracFC copia_etc            # i) mv también renombra
$ ls -d .*                       # j) (o ls -a)
```

d) Con el editor puedes modificar tu copia pero no `/etc/group`: es de root y no tienes permiso de escritura. Si copias con `-a` como usuario normal, el dueño de la copia pasa a ser tu usuario (no se puede «regalar» a root).

B1 · Ejercicios 5–8 · Copiar con el ratón, metadatos, historial y man

Guarda la salida de ls -l en salida.txt; compara su tamaño con ls -l y du -h; usa history, Ctrl-R y !num; consulta man ls.

```
$ ls -l salida.txt    # tamaño real en bytes (p. ej. 1239)
$ du -h salida.txt    # 4,0K: ocupa un bloque entero de 4 KiB
$ history             # lista numerada
$ !37                 # repite la orden 37 (Ctrl-R group busca la última
con «group»)
$ ls -a ; cat .bash_history
$ man ls              # /palabra busca, n la siguiente, q sale
```

ls da el tamaño lógico; du, los bloques que ocupa en disco.

B1 · Ejercicio 9 · Permisos sobre directorios

Pon a copia_etc los permisos 600, 500 y 000, prueba en cada caso cd y ls, y déjalo en 755.

```
$ chmod 600 copia_etc ; cd copia_etc   # Permiso denegado: falta x
$ chmod 500 copia_etc ; cd copia_etc ; ls ; cd   # r-x: entras y listas,
pero no puedes crear (sin w)
$ chmod 000 copia_etc ; ls copia_etc  # Permiso denegado: falta r
$ chmod 755 copia_etc
```

Octal	Permisos	cd	ls	crear dentro
600	rw-	✘	sólo nombres, con errores	✘ (sin x no se puede usar w)
500	r-x	✔	✔	✘
000	---	✘	✘	✘
755	rwx r-x r-x	✔	✔	✔ (el dueño)

B1 · Ejercicios 10-11 · Lanzar programas y borrar copia_etc

Lanza la calculadora de tres formas y borra el directorio copia_etc.

```
$ gnome-calculator      # desde el terminal (o kcalc); también desde el
menú o con Alt-F2
$ rm copia_etc          # rm: no se puede borrar 'copia_etc': Es un
directorio
$ rm -r copia_etc       # recursivo (añade -f para no preguntar)
```

Las aplicaciones de KDE funcionan en GNOME y al revés porque ambos corren sobre X-Window.

4.3 Boletín 2 resuelto

B2 · Ejercicio 1 · Búsquedas y comodines

Crea prueba/etc y prueba/include, copia allí los .h de /usr/include y lo de /etc que empieza por a, y usa find para mostrar: 1) directorios; 2) ficheros; 3) ficheros .h; 4) .h que empiezan por a–f; 5) ficheros que no son .h; 6) ficheros que empiezan por a–g. f) ficheros regulares de /etc, /usr/bin y /usr/sbin no accedidos en 7 días; g) ficheros de /etc entre 10 y 20 KiB.

```
$ mkdir -p prueba/etc prueba/include
$ cp /usr/include/*.h prueba/include
$ cp -r /etc/a* prueba/etc
$ find prueba                                # d) toda la jerarquía
$ find prueba -type d                        # 1)
$ find prueba -type f                        # 2)
$ find prueba -type f -name "*.h"            # 3)
$ find prueba -type f -name "[a-f]*.h"       # 4)
$ find prueba -type f ! -name "*.h"          # 5)
$ find prueba -type f -name "[a-g]*"         # 6)
$ find /etc /usr/bin /usr/sbin -type f -atime +7     # f)
$ find /etc -type f -size +10k -size -20k            # g)
```

Pon los patrones entre comillas para que los expanda find y no bash. Las búsquedas en /etc darán algunos «Permiso denegado»: añade `2>/dev/null` para ocultarlos.

B2 · Ejercicio 2 · Copias de seguridad con tar

Sin cambiar de directorio: copia prueba en copiaprueba con cp -Rp, crea prueba.tgz, compara tamaños, lista el .tgz, borra prueba y restáuralo.

```
$ cp -Rp prueba copiaprueba    # -R recursivo, -p conserva permisos,
dueño y fechas
$ tar czvf prueba.tgz prueba   # crear, comprimir, verbose, fichero
$ du -hs prueba ; ls -l prueba.tgz   # el .tgz ocupa bastante menos
$ tar tzvf prueba.tgz          # listar el contenido
$ rmdir prueba                 # falla: no está vacío
$ rm -rf prueba                # -r recursivo, -f sin preguntar
$ ls                           # ya no está
$ tar xzvf prueba.tgz ; ls prueba
```

B2 · Ejercicio 3 · Permiso de ejecución y PATH

Copia /bin/ls como myls, ejecútalo, quítale y devuélvele el permiso de ejecución.

```
$ cp /bin/ls myls
$ ./myls -l          # funciona
$ chmod -x myls      # (o chmod 644 myls)
$ ./myls             # bash: ./myls: Permiso denegado
$ chmod +x myls ; ./myls -l
$ echo $PATH         # el home no está en la lista: por eso hace falta ./
```

B2 · Ejercicio 4 · Primer y segundo plano

Lanza kcalc y kate en primer plano y en segundo, trae cada uno con fg, mata uno con Ctrl-C, duerme el otro con Ctrl-Z, despiértalo con bg y mátalo con kill -9.

```
$ kcalc ; kate ;        # kate no arranca hasta cerrar kcalc
$ kcalc & kate &        # [1] 4211  [2] 4212  (job y PID)
$ fg                    # trae kate (el último) → Ctrl-C lo mata
$ fg                    # trae kcalc → Ctrl-Z lo duerme (no responde)
$ bg                    # sigue en segundo plano y vuelve a responder
$ ps -Af | grep kcalc   # busca su PID si no lo apuntaste
$ kill -9 4211
```

B2 · Ejercicios 5-6 · Listar procesos y redirigir la salida

Con ps -Af localiza procesos tuyos y de root, el que más CPU ha usado y el último lanzado. Después guarda ls /bin en f1.txt y añade la fecha y el listado de /etc.

```
$ ps -Af           # UID = dueño, TIME = tiempo de CPU, STIME = hora de
inicio
$ ls /bin > f1.txt  # > crea o sobrescribe
$ cat f1.txt
$ date >> f1.txt    # >> añade al final
$ ls /etc >> f1.txt
$ gedit f1.txt &
```

El último proceso lanzado suele ser el propio `ps` (STIME más reciente); el de más CPU, el de mayor TIME (a menudo el navegador o el entorno gráfico).

5

Tema 5

Compilar, enlazar, depurar y traducir a ensamblador

Boletines 1 y 2 del tema 5: de hola.c al ejecutable, reubicación, gdb, y la traducción manual de C a ensamblador x86-64.

5.1 Del fuente al ejecutable

```
$ gcc48 hola.c -o hola                                          # compilar
todo
$ ./hola                                                        # Hola, mundo!
$ gcc48 hola.c -fno-asynchronous-unwind-tables -S -o hola.s     # ensamblador
$ gcc48 -c hola.c -o hola.o                                     # objeto
$ objdump -d hola.o > hola.o.disassembled                       # desensamblar
$ gcc48 hola.o -o hola                                          # enlazar (gcc
llama a ld)
$ ldd hola                                                      # librerías
dinámicas
$ gcc48 -static hola.o -o hola.static                           # ejecutable
estático
```

`objdump` muestra tres columnas: desplazamiento desde el comienzo del objeto, bytes del código máquina (longitud variable) y la instrucción. En el objeto, `bf 00 00 00 00` (mov $.LC0,%edi) y `e8 00 00 00 00` (call puts) tienen las direcciones a **cero**: todavía no se conocen.

`ldd hola` muestra `libc.so.6` (librería estándar de C, con `puts`), `linux-vdso.so.1` (llamadas al sistema) y `ld-linux-x86-64.so.2` (el cargador de librerías dinámicas). El dinámico ocupa ≈ 10 KB; el estático ≈ 800 KB y ldd dice que no es dinámico.

5.2 Reubicación y gdb

Con `-g` se añade información de depuración: `gdb hola.static`. Tras cargarlo, el código ya tiene direcciones virtuales definitivas (main en 0x400b4e) y las instrucciones reubicadas: `mov $0x491d04,%edi` (dirección de la cadena) y `callq 0x410200 <puts>`.

> **El call reubicado**
>
> Bytes `e8 a4 f6 00 00` en 0x400b57: el destino es relativo a la instrucción siguiente, 0x400b5c. 0x410200 − 0x400b5c = 0xF6A4 → en little endian y 32 bits: a4 f6 00 00.

Orden de gdb	Qué hace
`l` / `list main`	Lista el código C
`disassemble main`	Desensambla
`x/16bx main`	16 bytes en hex desde main
`x/13bc 0x491d04`	13 bytes como caracteres
`x/40bx array` · `p array`	Memoria del array / el array como en C
`break 8` · `b main` · `run`	Punto de ruptura y ejecutar
`next` · `step` · `stepi`	Sentencia (sin entrar / entrando en funciones) · instrucción
`info registers rip`	Ver un registro
`set array[5] = -2` · `set {int} 0x601064 = -2`	Modificar memoria
`disable` · `continue` · `backtrace`	Quitar breakpoints, seguir, pila de llamadas
`layout regs`	Vista de ensamblador y registros

En globals.c, `array` está en 0x601040, 4 bytes por entero en little endian. Tras `array[0] = -1` los primeros bytes son `ff ff ff ff` (−1 en C2). Al terminar el programa la memoria se resetea: pon un breakpoint en la llave final de main para ver el estado final.

5.3 Ejercicios del boletín 1

B1 · Ejercicio 1 · Dirección del último elemento

Ejecuta globals paso a paso con step observando i y array. ¿Cuál es la dirección del último elemento?

```
(gdb) b main
(gdb) run
(gdb) step          # repetir, y en cada vuelta:
(gdb) p i
(gdb) p array
(gdb) x/40bx array
```

ÚLTIMO ELEMENTO array[9] en 0x601040 + 9·4 = **0x601064**

En general, array[i] está en 0x601040 + 4i (con la dirección base que te dé tu gdb).

B1 · Ejercicios 2-3 · Sumar el array y modificar la memoria

Añade el bucle de suma con printf, ejecuta dos iteraciones viendo sum, pon array[5] = −2 y el último elemento a −2 con set {int}. ¿Qué suma sale?

> ⚠ **Cuidado**
>
> El código del boletín declara `int sum;` sin inicializar: su valor inicial es basura. Escribe `int sum = 0;` para que el resultado sea fiable.

```
(gdb) break 9          # línea del sum += array[i]
(gdb) run
(gdb) next ; print sum  # dos iteraciones: 10, 19
(gdb) set array[5] = -2
(gdb) x/40bx array      # array+20: fe ff ff ff
(gdb) set {int} 0x601064 = -2
(gdb) continue
```

La suma original es 10 + 9 + … + 1 = 55. Al cambiar array[5] (valía 5) y array[9] (valía 1) por −2 antes de sumarlos: 55 − 5 − 2 − 1 − 2 = **45**.

B1 · Ejercicio 4 · Array de short

Cambia el array a short int. ¿Qué tamaño tiene? ¿Dónde están el último y el i-ésimo elemento?

Cada short ocupa 2 bytes: el array mide **20 bytes** (`x/20bx array`). Con base B: array[i] en **B + 2i** y el último en **B + 18** (con B = 0x601040, 0x601052). Cada valor ocupa ahora 2 bytes en little endian: `0a 00 09 00 08 00…`.

B1 · Ejercicio 5 · Librería matemática estática y dinámica

Usa pow(3, i) con math.h y compila una versión estática y otra dinámica.

```
$ gcc48 -g -static globals.c -o globals.static -lm
$ gcc48 -g globals.c -o globals.dynamic -lm
$ ls -l globals.static globals.dynamic   # el estático, cientos de KB; el
dinámico, unos KB
$ ldd globals.static                     # no es un ejecutable dinámico
$ ldd globals.dynamic                    # libm.so.6, libc.so.6, ld-
linux…
```

`-lm` enlaza la librería matemática (libm), donde está pow.

5.4 Traducción manual de C a ensamblador

Con `-O0` el compilador genera mucho código (17 instrucciones en aritmetica.c, leyendo y escribiendo myVar cuatro veces); con `-O1`, 4. El boletín usa traducciones hechas a mano, más legibles. El valor devuelto por main se ve con `echo $?` (sus 8 bits bajos).

aritmetica_manual.s

```
        .data
myVar:  .long 3            # int myVar = 3;
        .text
        .globl main
main:   movl  myVar, %eax  # EAX = 3
        addl  $5, %eax     # 8
        sall  $2, %eax     # 32  (desplazar 2 = ×4)
        movl  $2, %edx
        imull %edx         # EDX:EAX = EAX × EDX = 64
        decl  %eax         # 63
        movl  %eax, myVar
        ret                # devuelve 63
```

funcion_manual.s

```
funcion_resta:              # parámetros: EDI = minuendo, ESI = sustraendo
        movl  %edi, %eax
        subl  %esi, %eax    # EAX = EDI - ESI (valor devuelto)
        ret
main:   movl  $100, %ebx    # resultado = 100
        movl  $50, %edi     # 1.er argumento
        movl  $30, %esi     # 2.º argumento
        call  funcion_resta # EAX = 20
        addl  %ebx, %eax    # 120
        inc   %eax          # 121
        ret
```

El bucle de array_manual.s está en la guía de teoría (5.3). Convención System V AMD64: argumentos en RDI, RSI, RDX, RCX, R8, R9 y resultado en RAX (con int se usan las mitades E...).

5.5 Ejercicios del boletín 2

B2 · Ejercicio 1 · aritmetica_manual paso a paso

Ensámblalo con -g, cárgalo en gdb con layout regs, mira myVar y ejecuta con stepi.

```
$ gcc48 -g aritmetica_manual.s -o aritmetica_manual
$ gdb aritmetica_manual
(gdb) layout regs
(gdb) print &myVar          # o búscala en el disassemble
(gdb) x/4bx &myVar          # 03 00 00 00
(gdb) print (int)myVar      # 3
(gdb) b main
(gdb) run
(gdb) stepi                 # repetir mirando EAX y EDX
```

Tras...	EAX	EDX	myVar
movl myVar,%eax	3	—	3
addl $5	8	—	3
sall $2	32	—	3
movl $2,%edx	32	2	3
imull %edx	64	0	3
decl %eax	63	0	3
movl %eax,myVar	63	0	63

`imull %edx` deja el producto de 64 bits en EDX:EAX: EDX pasa a 0.

B2 · Ejercicio 2 · array_manual: ¿dónde queda la comparación?

Ejecuta array_manual con stepi viendo la memoria del array y los registros. ¿En qué registro se guarda el resultado de la comparación?

En cada vuelta `movl array(,%esi,4), %ecx` carga array[i] en ECX y `addl` lo acumula en EAX (10, 30, 60, 100, 150). El resultado de `cmpl $5, %esi` no va a un registro de datos: se guarda en los **flags del registro RFLAGS (EFLAGS)**, que es lo que consulta `jge`. El programa devuelve 150.

B2 · Ejercicio 3 · call y ret con RIP y RSP

Mira RIP, RSP y x/1qx $rsp antes y después de cada call y ret. ¿Qué hacen? ¿Qué pasaría si la dirección de retorno se guardara en un registro?

Instrucción	RSP	Memoria	RIP
call f	RSP − 8	En [RSP] queda la dirección de la instrucción siguiente al call	dirección de f
ret	RSP + 8	Lee la dirección de retorno de [RSP]	esa dirección

La dirección de retorno se guarda en la **pila** y ret la saca de ahí. Si se guardara en un registro, una función que llame a otra (o una recursiva) la **machacaría** y no podría volver; habría que salvarla a mano en la pila (es lo que se hace en MIPS con $ra).

B2 · Ejercicios 4–5 · Desbordamiento de pila

Ejecuta stackoverflow y sigue su pila con pmap y con backtrace en gdb. ¿Qué ocurre y qué tamaño máximo alcanza?

```
$ pmap <PID>                  # en otro terminal: la zona [ stack ] crece
$ gdb stackoverflowsimple
(gdb) b ping
(gdb) run
(gdb) continue ; backtrace    # repetir hasta el marco 10
(gdb) disable ; continue      # hasta que reviente
(gdb) backtrace -1            # el marco más externo
$ ulimit -a                   # stack size: normalmente 8192 KB
```

Es una recursión sin fin: cada llamada añade un marco (dirección de retorno, RBP, locales) y la pila crece hasta superar el límite de `ulimit -s` (típicamente 8 MB). El SO mata el proceso con **Segmentation fault**: es un **stack overflow**.

B2 · Ejercicios 6–7 · Parámetros y variables locales

Observa EDI, ESI y EAX en funcion_manual. En locales, compara el tamaño de la pila con pmap al meter un número cualquiera y el 1234. ¿Dónde están inputNum y magicNum?

Antes del call, EDI = 50 y ESI = 30; al volver, EAX = 20 y después 121 (`echo $?`).

En locales, con el número mágico se ejecuta una función distinta (mira el main del fuente) que reserva más variables locales o hace más llamadas, y pmap muestra la zona de pila más grande. `inputNum` y `magicNum` son variables **locales**, así que están en la **pila** (en el marco de su función), no en el segmento de datos.

6

Tema 6

Configuración de red, DNS y Web

Boletín 1 del tema 6: consultar la configuración IP, comprobar la conexión, el DNS con host y el protocolo HTTP a mano con telnet y nc.

6.1 Configuración IP: ifconfig, route y ping

`ifconfig` (sin parámetros) muestra las interfaces activas: `eth0` / `wlan0` (Ethernet/WiFi), `lo` (loopback 127.0.0.1, el equipo hablándose a sí mismo) y otras de virtualización (docker0, virbr0).

ifconfig (fragmento)

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
      inet 155.54.223.216  netmask 255.255.255.224  broadcast 155.54.223.223
      ether 54:a0:50:7a:f8:8c
```

IP 155.54.223.216 con máscara 255.255.255.224 (/27) → subred **155.54.223.192/27**, broadcast .223. La **MAC** (dirección hardware) es 54:a0:50:7a:f8:8c.

route -n

```
Destino         Pasarela        Genmask          Indic Interfaz
0.0.0.0         155.54.223.221  0.0.0.0          UG    eth0
155.54.223.192  0.0.0.0         255.255.255.224  U     eth0
```

La red 155.54.223.192/27 se alcanza directamente (pasarela 0.0.0.0) y todo lo demás (destino 0.0.0.0) va al **router 155.54.223.221**. `ping host` envía paquetes y mide el tiempo de ida y vuelta: sirve para saber si un equipo responde.

6.2 Herramienta: calculadora de red INTERACTIVO

Escribe una IP con su prefijo (o con su máscara) y otra IP para saber si están en la misma red.

6.3 DNS, HTTP, telnet, nc y netstat

El **DNS** es un protocolo de aplicación cliente-servidor sobre **UDP, puerto 53**, que traduce nombres en IP. Al escribir www.um.es: el navegador pasa el nombre al cliente DNS, éste pregunta al servidor DNS primario (o al secundario), recibe la IP y el navegador abre una conexión TCP al puerto 80 de esa IP. También hace traducciones inversas, alias y balanceo de carga.

```
$ host www.um.es
www.um.es is an alias for wwwclu.um.es.
wwwclu.um.es has address 155.54.212.103
$ host 155.54.212.103          # inversa: nombres asociados a la IP
```

HTTP: sobre TCP, sin estado. Se puede hablar con un servidor a mano con `telnet servidor 80` escribiendo la petición y una línea en blanco. Códigos: 200 OK, 301 Moved permanently, 400 Bad Request, 404 Not Found.

`netstat -tupan` lista conexiones TCP (-t) y UDP (-u), también las que están en escucha (-a), con números de puerto (-n) y el PID/programa (-p).

6.4 Ejercicios resueltos

Ejercicio 1 · Configuración IP del PC

Averigua la IP, la máscara, el router por defecto y los servidores DNS.

```
$ ifconfig            # inet (IP) y netmask de eth0
$ route -n            # la fila con destino 0.0.0.0 da el router
(pasarela)
$ nmcli dev show      # IP4.DNS[1], IP4.DNS[2]: servidores DNS
```

Con los datos del ejemplo: 155.54.223.216/27 en la red 155.54.223.192, router 155.54.223.221. Si no tienes ifconfig, `ip addr` e `ip route` dan lo mismo.

Ejercicio 2 · Comprobar la conexión con ping

Haz ping a un equipo de tu red, al router, a www.um.es y a www.google.es.

```
$ ping -c 3 155.54.223.217    # otro equipo de la subred
$ ping -c 3 155.54.223.221    # el router
$ ping -c 3 www.um.es         # usa el DNS para obtener la IP
$ ping -c 3 www.google.es
```

Cada respuesta muestra el tiempo (time=... ms); al final, paquetes enviados, recibidos y perdidos. Sin `-c` sigue hasta pulsar Ctrl-C. Algunos servidores no responden al ping aunque estén funcionando.

Ejercicio 3 · Consultas con host

a) IP de www.carm.es; b) ¿es www.wikipedia.es un alias?; c) cuántas IP tiene www.hotmail.com; d) nombre de 155.54.212.102.

```
$ host www.carm.es            # «has address …»
$ host www.wikipedia.es        # si aparece «is an alias for …», es un
alias
$ host www.hotmail.com         # cuenta las líneas «has address»
$ host 155.54.212.102          # «domain name pointer …»
```

Las respuestas concretas cambian con el tiempo (los servicios grandes cambian de IP y tienen varias para repartir la carga), así que fíjate en el formato: **has address** = IP, **is an alias for** = alias, **domain name pointer** = traducción inversa.

Ejercicios 4–5 · Navegar por nombre y por IP

Abre slashdot.org en el navegador y después con su dirección IP.

Obtén la IP con `host slashdot.org` y escribe `http://IP/`. Puede que funcione o que el servidor responda con un error o una redirección: un mismo servidor (o una CDN) aloja muchos sitios y decide cuál servir por la cabecera `Host:`, que al usar la IP ya no lleva el nombre.

Ejercicio 6 · HTTP a mano con telnet

Con telnet: a) pide la página principal de www.ditec.um.es; b) pide una que no existe; c) pide ~pedroe/documentos/prueba-con-imagen.html y luego su imagen.

```
$ telnet www.ditec.um.es 80
GET / HTTP/1.1
Host: www.ditec.um.es
Connection: close
                                  ← línea en blanco: fin de la petición
HTTP/1.1 200 OK …                 ← a) cabeceras + HTML

GET /~pedroe/documentos/fallo.html HTTP/1.1     ← b) en otra conexión
Host: www.ditec.um.es
                                  → HTTP/1.1 404 Not Found
```

c) Pide la página, busca en su HTML el `<IMG SRC="…">` y abre otra conexión con `GET` a esa ruta: la respuesta tiene `Content-Type: image/…` y datos binarios. Hay que escribir rápido: el servidor cierra la conexión si tardas. Si el servidor redirige a HTTPS (301), es normal hoy en día.

Ejercicio 7 · Hacer de servidor web con nc

Escucha en el puerto 30001 con nc, abre http://TU_IP:30001 en el navegador y contesta «Hola» como text/plain.

```
$ nc -l 30001          # (o nc -l -p 30001)
GET / HTTP/1.1         ← llega la petición del navegador
Host: 155.54.223.216:30001
User-Agent: Mozilla/5.0 …

HTTP/1.1 200 OK        ← lo escribes tú
Content-Type: text/plain
Content-Length: 4

Hola
```

Tras la línea en blanco van exactamente `Content-Length` bytes. Para responder HTML usa `Content-Type: text/html` y cuenta bien los bytes. Pulsa Ctrl-C en nc para cerrar si el navegador se queda esperando.

Ejercicios 8-9 · netstat y la tabla de rutas

Repite algún ejercicio mirando netstat -tupan e interpreta route y route -n.

Con el navegador abierto verás líneas `tcp … 155.54.223.216:34020 93.184.220.29:80 ESTABLECIDO 3424/firefox`: tu IP y un **puerto de origen alto**, la IP remota y el **puerto 80 o 443**. Mientras `nc -l 30001` espera, aparece `0.0.0.0:30001 … ESCUCHAR`.

`route` sin `-n` muestra nombres (default, la pasarela por nombre) y tarda más porque consulta el DNS; `-n` muestra números. Interpretación: la fila **UG** con destino 0.0.0.0 es la ruta por defecto (U = activa, G = por un gateway); las filas **U** son redes conectadas directamente.

Basado en los boletines de prácticas de la asignatura. Los bytes, las direcciones y las funciones lógicas de las resoluciones están comprobados por ordenador.